ds_finder

Minimum Dominating Set — PACE 2025

Hamza Konte — L3 MIAGE APP

Université Paris-Dauphine PSL — Février 2026

VS

Contrôle mémoire

Structure minimaliste

Zéro abstraction

```c
typedef struct s_node {
    int *neighbors;
    int  degree;
    int  capacity;
} t_node;

typedef struct s_graph {
    int          v_count;
    int          e_count;
    t_graph_type type;
    t_node      *nodes;
    t_bool      *solutions;
    int          len_solutions;
} t_graph;
```

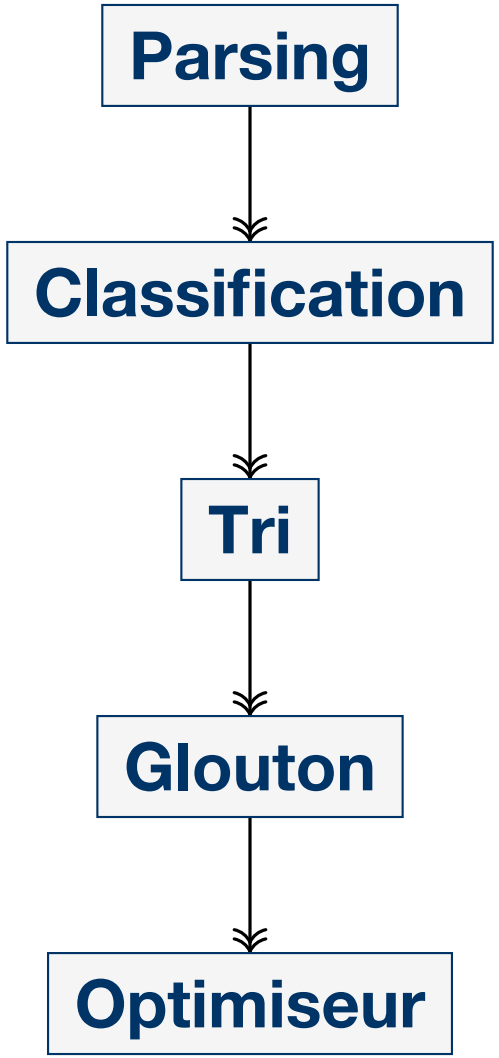
`covers[u]` = u dominé par combien

`solutions[u]` = u ∈ D

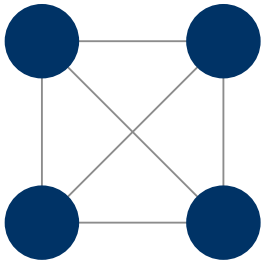
`actives[u]` = non encore dominé

Tableaux bruts | Buffers pré-alloués | `t_bool` = 1 octet

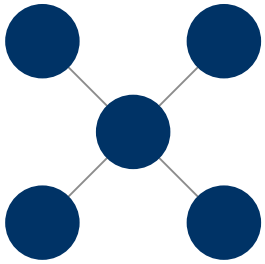
Pipeline de résolution



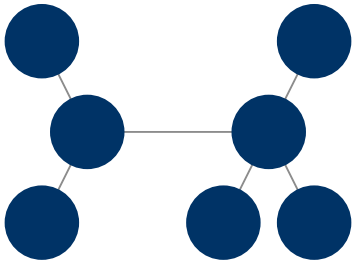
Classification empirique par métriques (1/2)



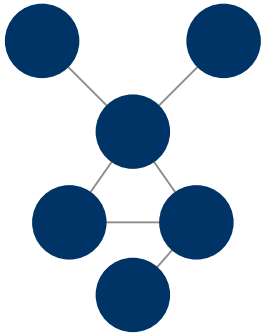
Complet



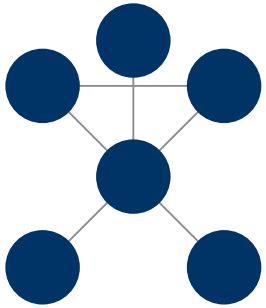
Étoile



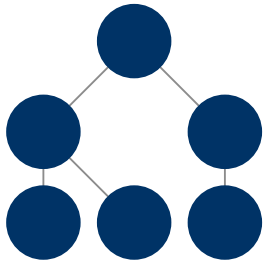
Multi-étoile



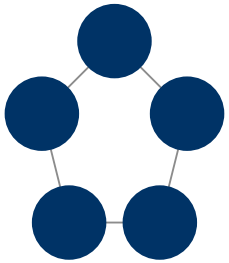
Core-périph.



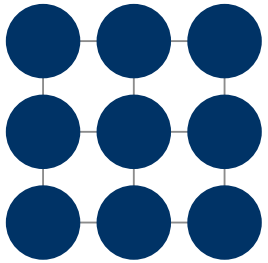
Scale-free



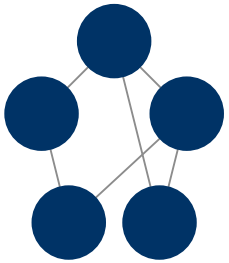
Arbre



Anneau



Grille 2D



Erdős–Rényi

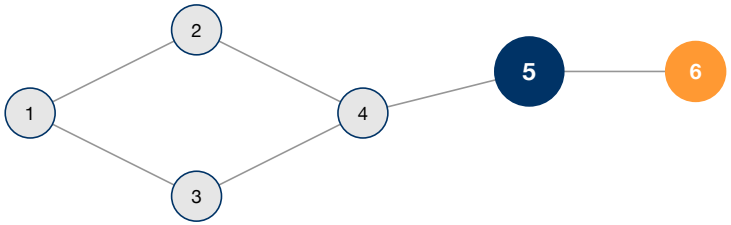
?

Autre

8 types ⟶ **Glouton classique**

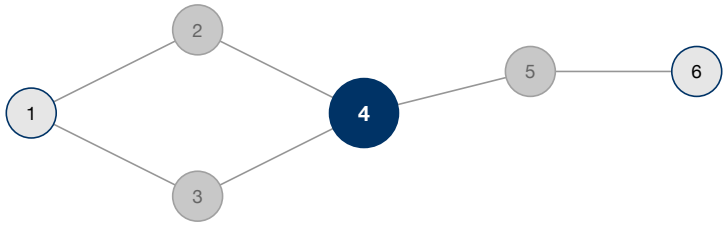
Core-périph. + Grille ⟶ **Glouton adaptatif**

Phase feuilles : le voisin de chaque feuille est ajouté à D



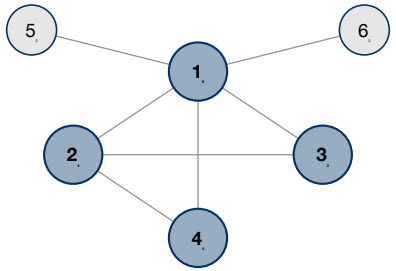
Complexité : $O(V + E)$

Le nœud de plus haut degré est ajouté à D



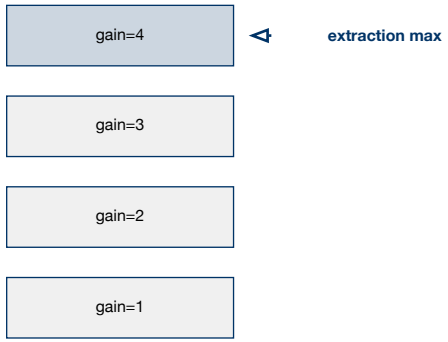
Itérer jusqu'à couverture complète

$$\text{gain}(u) = 1 + |\{v \in N(u) : \text{covers}[v] = 0\}|$$



Le gain évolue dynamiquement → recalcul à chaque extraction

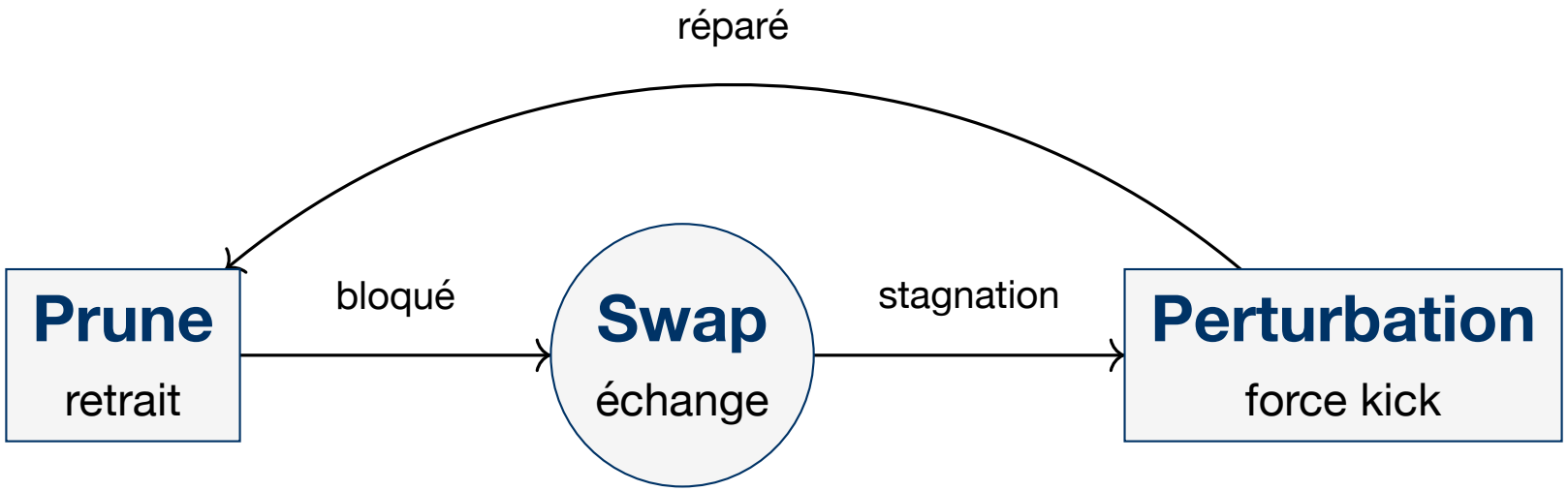
Bucket sort + lazy evaluation



Si gain réel ≥ gain bucket : → ajouter à D ✓

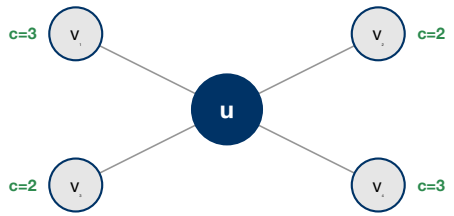
Sinon : re-bucketing ↓ → replacer au bon niveau

$O(V^2 + E)$ pire cas
$O(V \log V)$ en pratique

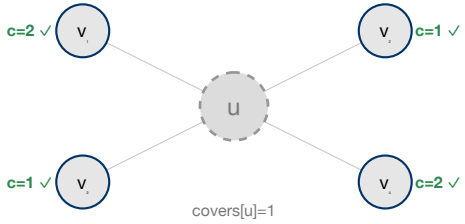


Itère jusqu'à SIGTERM — sauvegarde la meilleure solution à chaque amélioration

Prune (1→0) — Condition

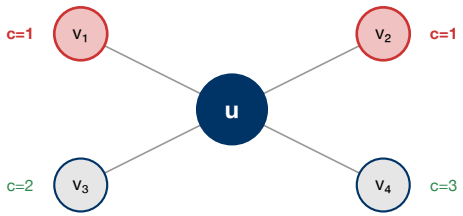


Condition : `covers[u] ≥ 2` ET `∀v ∈ N(u), covers[v] ≥ 2` ⟶ **u retirable**



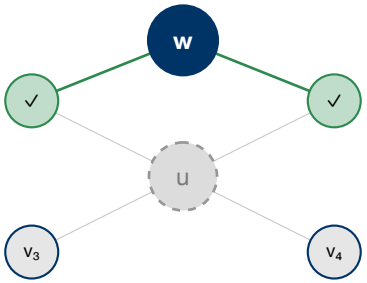
|D| diminue de 1 — Complexité : $O(\deg(u))$

Swap (1→1) — Identifier



Voisins privés : couverts UNIQUEMENT par u

Swap — Échanger



|D| inchangé — Tabu : $T = 1 + \mathrm{rand}(0..9)$

1. Stagnation

`lock_count ≥ 2`

Aucune amélioration

2. Éjection

u aléatoire retiré

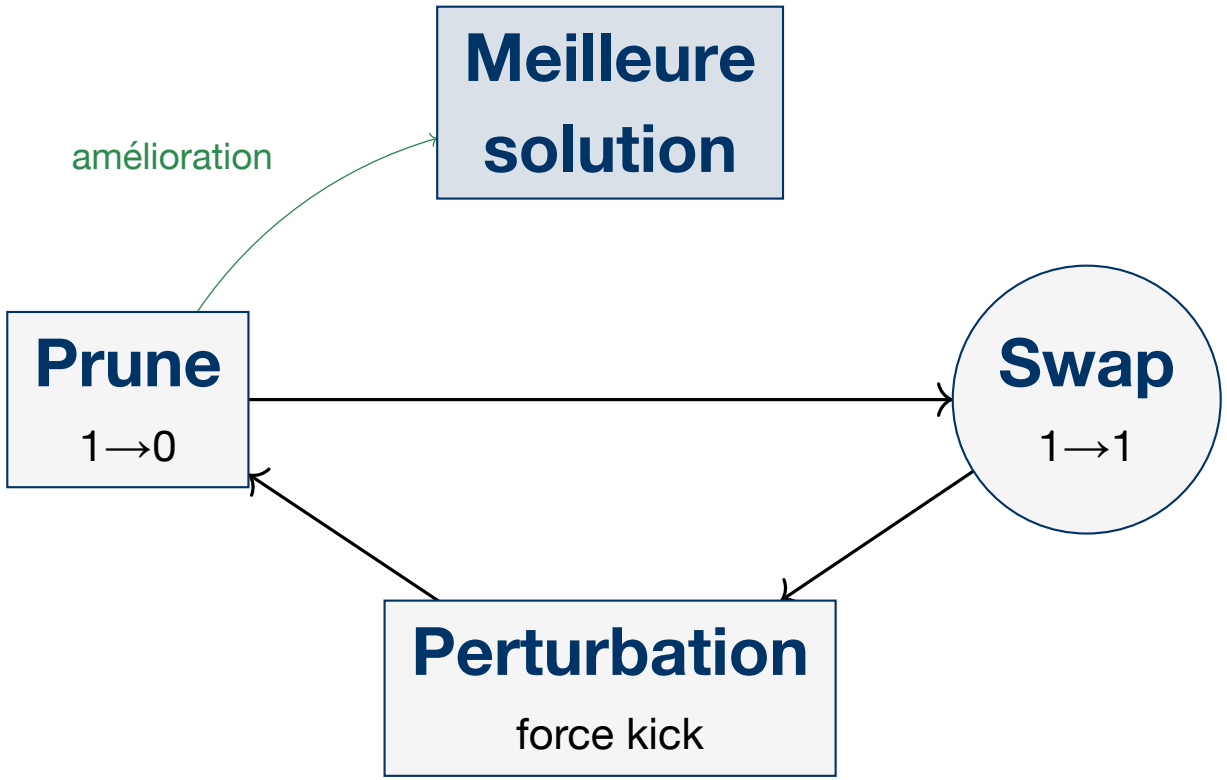
Nœuds non couverts !

3. Réparation

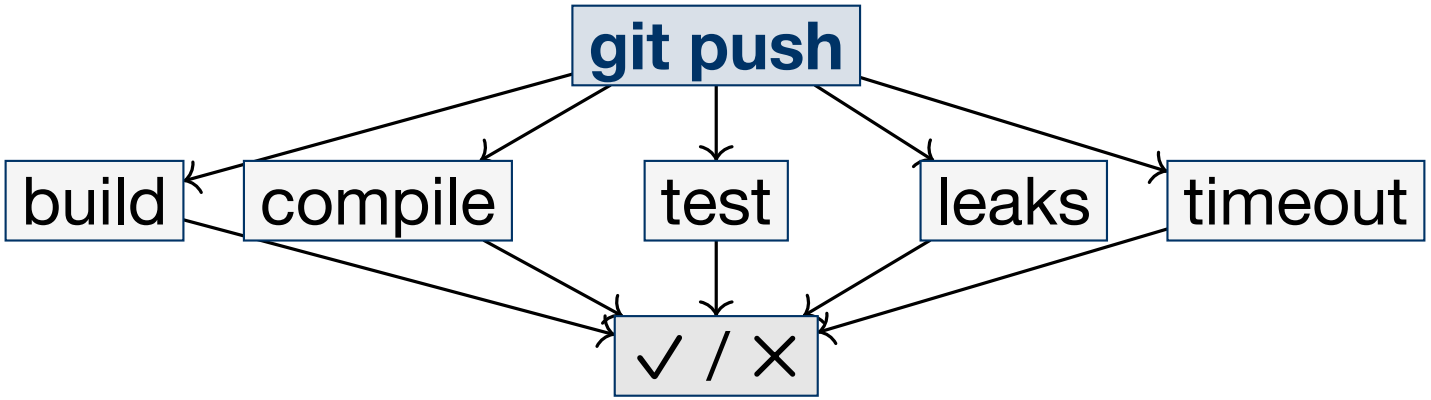
Voisins manquants → D

Couverture rétablie

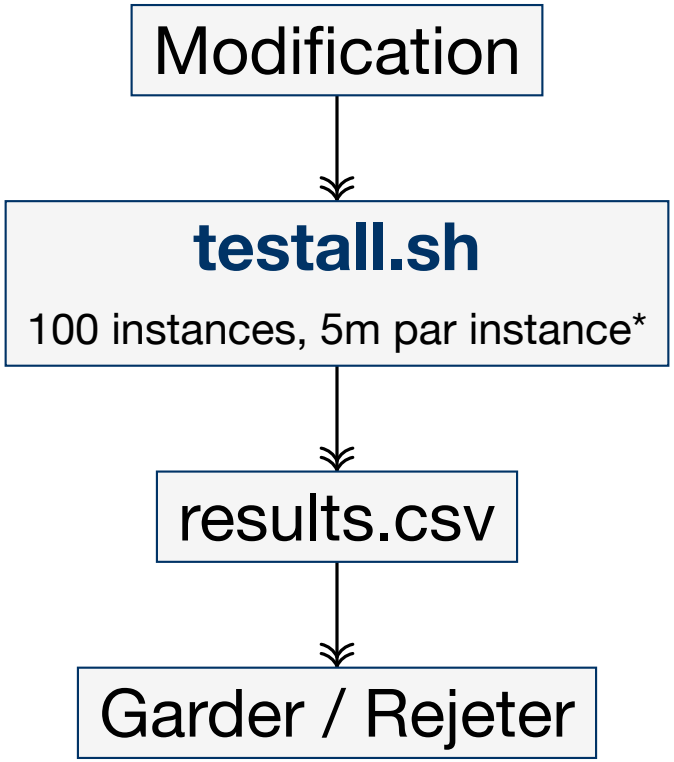
Tabu(u) = 20 itérations — empêche la réinsertion immédiate



Complexité : $O\left(I \times V \times \overline{d}\right)$ — borné par le temps



160+ commits — chaque push vérifié automatiquement


Modification
testall.sh
100 instances, 5m par instance*
results.csv
Garder / Rejeter

Gardé ✓

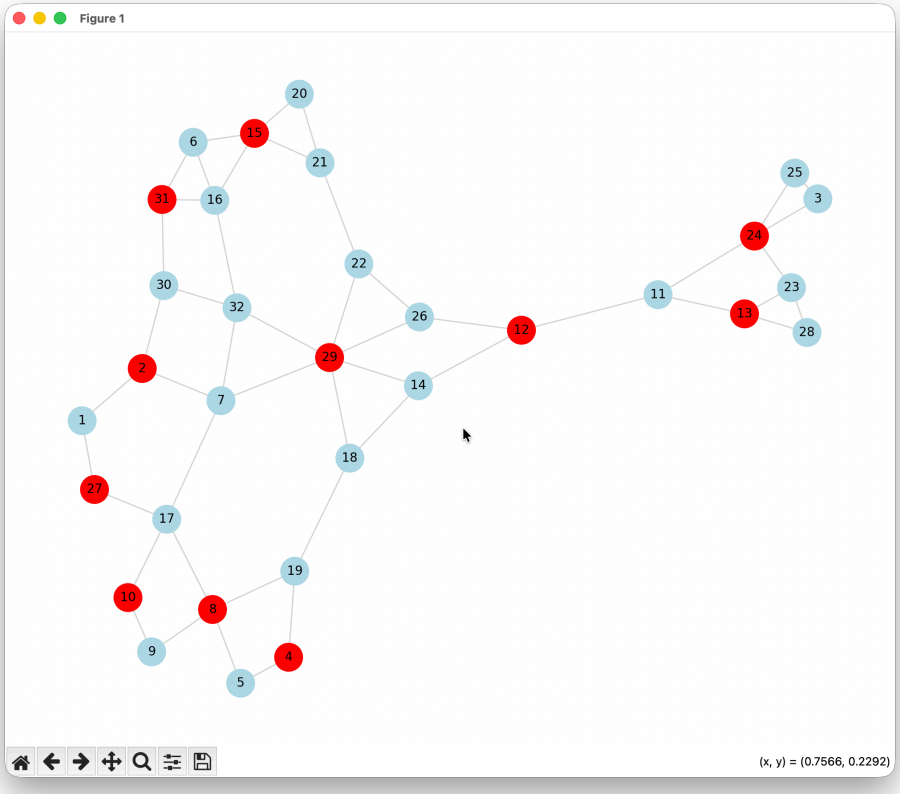
- ✓ Tri par degré
- ✓ Core-périph.
- ✓ Swap
- ✓ Early exit p>15
- ✓ -O3 -flto
- ✓ Classification
- ✓ Prune
- ✓ Perturbation
- ✓ XOR RNG
- ✓ Tabu tenure=1

Rejeté ×

- × Kernelisation
- × Tenure=2
- × Tenure=0
- × CSR compaction
- × Cascade prune
- × Freq-guided ejection
- × Dynamic greedy repair

Chaque décision validée par A/B testing sur 100 instances

Visualisation



800
lignes de C

160+
commits

3 mois
de développement

5 jobs CI par push

99.96 sur Optil.io

Forces

- Bon sur les instances à **DS élevé**
- Grands graphes → bonnes solutions relatives
- Pipeline robuste et vérifié par CI
- Démarche empirique systématique

Limites

- Mauvais sur les instances à **DS faible**
- Petits DS optimaux → l'heuristique surdimensionne
- Perturbation trop peu agressive

IA ?

Merci

Questions ?